

MODULE 8

Components and routing

CourseCard · imports[] · @for + track · lazy routes · :id → input()

TMS client

```
course = input.required<Course>();  
enrollClicked = output<Course>();  
  
[course]="myCourse"  
(enrollClicked)="handleEnroll($event)"
```

Split the UI like you split the API

Session goals

By the end of this session, you will:

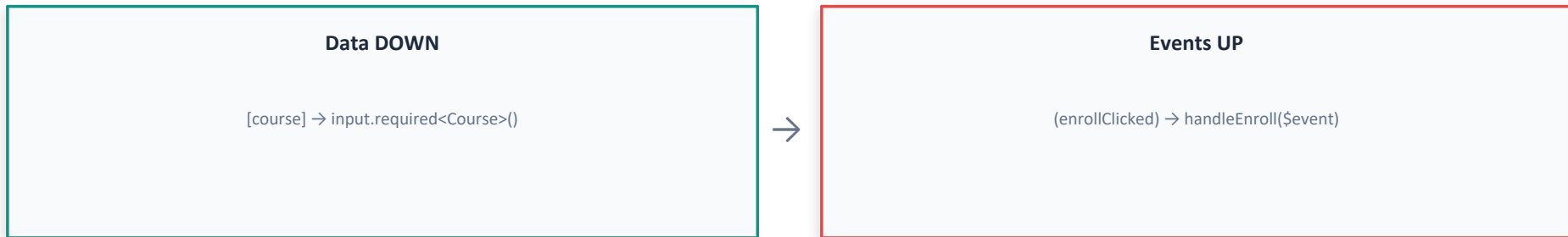
- Split StudentDashboard into CourseCardComponent data down, events up
- Declare `input.required<Course>()` and `output<Course>()`; wire `[course]` and `(enrollClicked)`
- Fix the import trap child class in parent `imports[]` or the tag is dead text
- List courses with `@for (...; track course.id)` and `@empty`
- Lazy-load routes with `loadComponent + dynamic import()`; keep initial bundle small
- Bind `/courses/:id` to `CourseDetailComponent` via `withComponentInputBinding + input name match`

The problem with one big component

Right now, your dashboard component is doing everything: holding the student data, defining the course array, rendering every card, and handling every click event. That works with 50 lines. It does not work with 500.

In the backend, you separated controllers from services from repositories. The frontend has the same principle: break UI into focused, reusable components.

Parent-child communication



- StudentDashboardComponent owns lists and handlers; CourseCardComponent owns one row of UI
- Contract mental model like a .NET interface: child declares what it needs; compiler enforces it

Course type matches Module 6

The `Course` interface in the lab workbook mirrors `CourseResponseDto` (camelCase JSON): numeric `id`, `code`, `title`, `maxCapacity`, `enrollmentCount`. Session 2 mock rows use that shape so `track course.id` is the real primary key and Lab 6 maps `PagedResponse.items` into the same `Course[]` your cards already bind. `Full` means `enrollmentCount >= maxCapacity` same idea as capacity on the API.

CourseCardComponent the contract (TypeScript)

course-card.ts (pattern)

```
// Child component strict contract (spine)

export class CourseCard {

  // I need a Course to render. Give me one.

  course = input.required<Course>();

  // When user clicks Enroll, I emit which course.

  enrollClicked = output<Course>();

}
```

input.required

Parent that forgets [course] does not compile not optional sugar.

Parent uses the card (template)

student-dashboard template (spine)

```
<tms-course-card
```

```
  [course]="myCourse"
```

```
  (enrollClicked)="handleEnroll($event)"
```

```
/>
```

- Square brackets = property binding (data into child)
- Parentheses prefix on event name = listen for child's output
- \$event carries the Course payload from enrollClicked.emit(course)

The import trap

Parent @Component (spine)

```
@Component({  
  selector: 'app-student-dashboard',  
  standalone: true,  
  imports: [CourseCard], // spine: without this, tag is ignored  
  templateUrl: './student-dashboard.html'  
})
```

Failure mode

Forget imports[] → <tms-course-card> shows as raw text in the DOM no Angular error. Demo before/after on purpose.

Looping with @for

Template (spine)

```
<div class="grid">

  @for (course of availableCourses(); track course.id) {

    <tms-course-card

      [course]="course"

      (enrollClicked)="handleEnroll($event)"

    />

  } @empty {

    <p>No courses found for this term.</p>

  }
```

- track is mandatory omit it and the compiler refuses to build
- @empty gives honest UX when the signal array has no rows

Then: Why track matters

Next slide without the right track expression, list changes can destroy and recreate DOM nodes instead of moving the cards you already built (spine).

Why track matters

Track strategy	When the array reorders	Performance
track course.id	Angular moves existing DOM nodes	Fast
track \$index	Angular thinks every item changed (index 0 is now a different course)	Slow destroys and recreates
Omitted	Compiler error app will not build	N/A

Whiteboard drill

Without track, every DOM node can churn when the array changes; with track course.id, Angular moves, adds, or removes only what changed (spine).

Array [A, B, C] → [C, A, B]. With id: three moves. With \$index: six destroy/create operations spine speaker note.

Routing URLs map to components

app.routes.ts (spine)

```
export const routes: Routes = [

  {
    path: 'dashboard',

    loadComponent: () => import('./features/student-dashboard/student-dashboard')
      .then(m => m.StudentDashboard)
  },

  {
    path: 'courses/:id',

    loadComponent: () => import('./features/course-detail/course-detail')
      .then(m => m.CourseDetail)
  },

  { path: '', redirectTo: 'dashboard', pathMatch: 'full' }

];
```

- Lazy loading with loadComponent + import(), the browser downloads that route's JS only when the user navigates there (spine)
- Liya's browser does not download admin-only route chunks
- Admin does not download enrollment-heavy chunks until they navigate to those routes (same spine idea as separate apps)

The input binding shortcut (:id)

course-detail.ts (spine)

```
// app.config.ts already: withComponentInputBinding()

export class CourseDetail{

  id = input.required<string>(); // "42" from /courses/42

}
```

- No `ActivatedRoute.subscribe` no manual unsubscribe
- Input property name must match route param exactly `:courseId` needs `courseId`, not `id`
- Facilitator beat: let the room enjoy "that's it?" then warn on param naming before undefined `id()` burns lab time

Common mistakes Session 2

Mistake	What you see	How to fix
Missing child component in `imports` array	The <code><tms-course-card></code> tag appears as raw text	Add <code>'CourseCard'</code> to the parent's <code>`imports`</code>
Missing <code><router-outlet /></code>	URL changes, page goes blank	Add the outlet to <code>`app.html`</code>
Tracking by <code>\$index</code> on filtered lists	List visually "jumps" when items are added/removed	Track by the database ID (track <code>course.id</code>)
Input name does not match route parameter	<code>id()</code> returns undefined on the detail page	Match the route param name exactly

Session 2 summary

- Compose TMS UI cards are contracts: input down, output up
- `imports[]` is the silent killer raw tag text means you forgot the child class
- `@for` requires track prefer `course.id` over `$index` for real TMS lists
- `loadComponent` keeps Liya's first paint small; route params flow to `input()` when names match

Next: Session 3 · Forms + live API (one thematic session)

Part A: Reactive Forms (Lab 5, Tier 4). Part B: HttpClient, rxResource, CORS-aware reads, POST enrolment flow (Lab 6, Tier 5) still on `tms-client`.